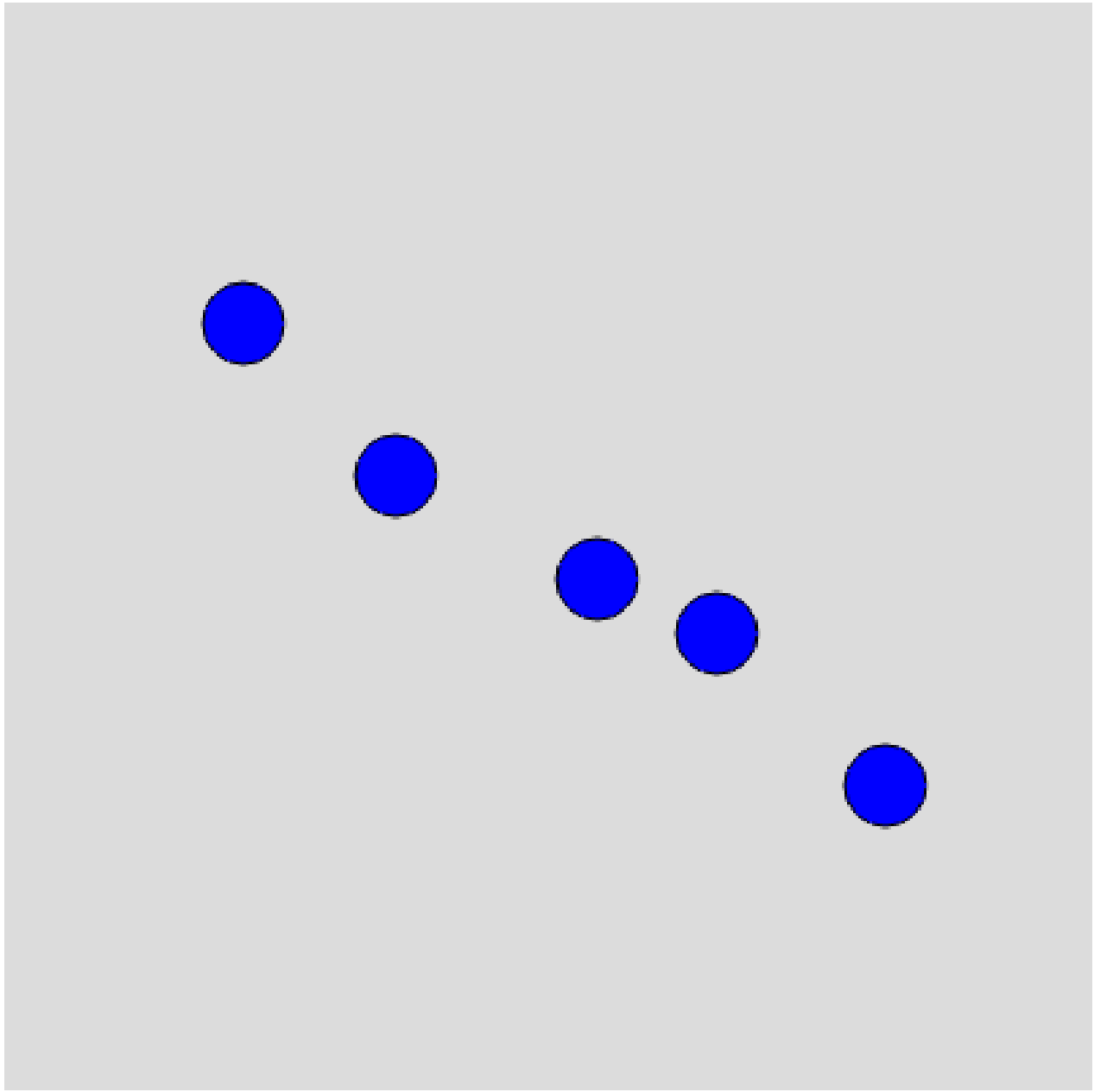
Creative Programming: Fundamentals

Lecture 2 (3 Feb)

Narration

Table of contents

Narration



Narration

(Re)visiting slides?

You can jump to any slide with the menu to the (bottom) left.

Did you miss a slide or want to revisit?
Open the narration tab while
studying to get an explanation of
difficult slides.

Narration

Handing in exercises

- Programming is all about practice, so that is what the exercises are all about
- You submit on LearnIT the things you want feedback on
- There is a hand-in instruction document available on LearnIT

Narration

Reference, System Variables, and Recap

Narration

Do programmers know everything???

- Programming is about knowing how to express yourself in a programming language …
- … but you do not need to remember all functions!

Narration

p5js Reference

- Want to draw a circle, but unsure about the arguments? -> Check the p5js reference
- Don’t know how to change the background colour? -> Check the p5js reference

Narration

Slides reference

- The slides also contain code examples for many p5js functions
- This week will be very code-heavy, so please refer to the slides and code examples often!
- All code examples are available on GitHub as well. There are over 20 for just this week!

Narration

mouseX and mouseY

- Last week we briefly introduced mouseX and mouseY, but what are they exactly?
- According to the p5js reference:
  - A Number **system variable** that tracks the mouse's horizontal position.

Narration

System Variables

- Named containers that the system manages for us
- Created and managed automatically by p5js

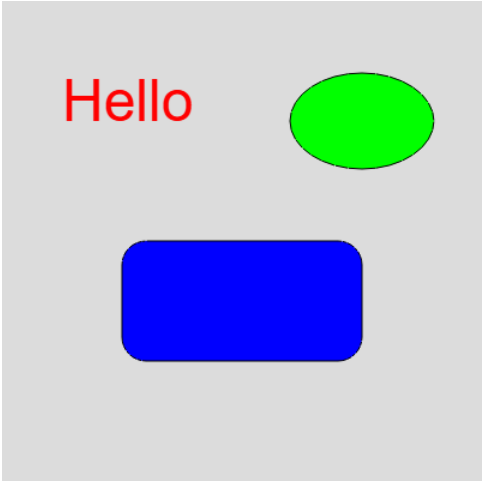
- Examples:
  - **mouseX**: horizontal position of the mouse
  - **mouseY**: vertical position of the mouse
  - **width**: horizontal size of the canvas
  - **height**: vertical size of the canvas
  - **frameCount**: amount of times draw() has been called since start

Narration

Recap: Shapes

- `text()`, `ellipse()`, and `rect()`
- You need to specify the coordinates (`x`, `y`) and the size / contents

```
1  function setup() {
2      createCanvas(400, 400);
3      background(220); // grey background
4
5      // Set text properties
6      textSize(48); // big text
7      fill(255, 0, 0); // red colour
8      text("Hello", 50, 100);
9
10     // Draw an ellipse
11     fill(0, 255, 0); // green colour
12     ellipse(300, 100, 120, 80);
13
14     // Draw a rounded rectangle
15     fill(0, 0, 255); // blue colour
16     rect(100, 200, 200, 100, 20); // The 5th argument is *optional* and defines the corner
```

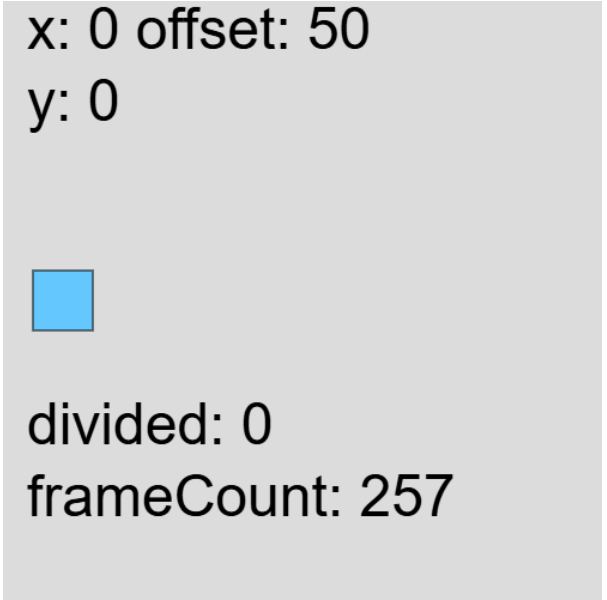


Narration

Mathing out

- We can use operators such as `+`, `-`, `*`, and `/`.
- This allows you to e.g. add an offset to the mouse position
- `+` can also add strings together

```
function setup() {
  createCanvas(500, 500);
  textSize(48);
}

function draw() {
  background(220);

  fill(100, 200, 255); // rect colour
  // x is 25 pixels to the right of the mouse
  // y is centered vertically
  // we use the 25 to center the rectangle due to its 50 pixel width and height
  rect(mouseX + 50 - 25, height / 2 - 25, 50, 50);
```



Narration

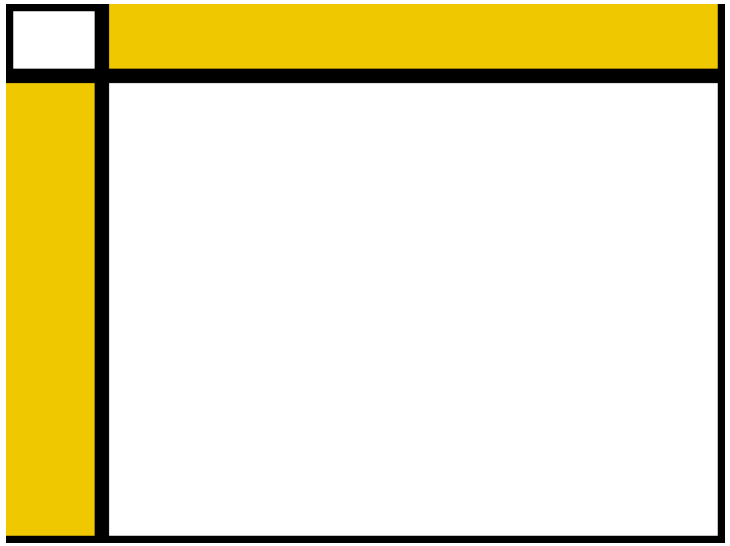
Mondrian

Narration

System Variables + Shapes

- With a bit of creativity we can make an interactive ‘Mondrian’ painting!

```
1  function setup() {
2    createCanvas(600, 450);
3  }
4
5  function draw() {
6    stroke(0);
7    strokeWeight(12);
8
9    // Top-left
10   fill(255, 255, 255);
11   rect(0, 0, width, height);
12
13   // Top-middle
14   fill(0,0,255);
15   rect(mouseX - 80, 0, 160, mouseY - 60);
16
17   // Middle-left
18   fill(255,0,0);
```



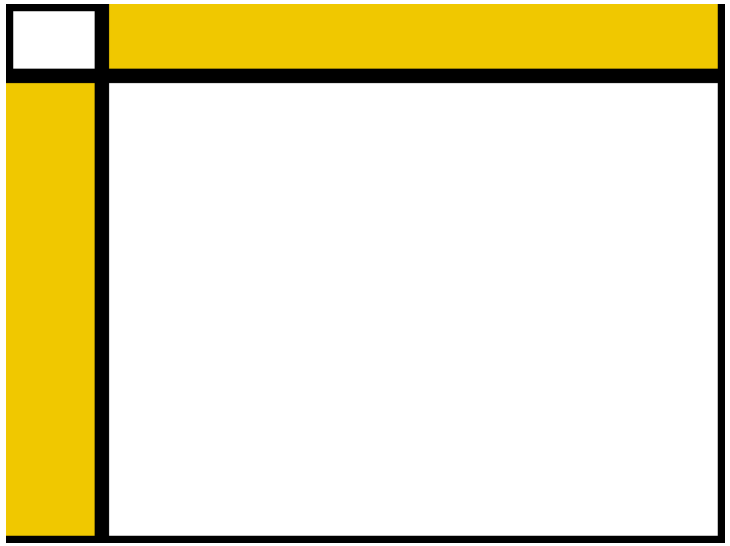
Narration

Your own Mondrian

Class Assignment

Create your own interactive Mondrian painting! In groups of 1-3, make a p5js sketch that draws rectangles (or other shapes) based on the mouse position. Be creative with colors and shapes!

```
1  function setup() {
2    createCanvas(600, 450);
3  }
4
5  function draw() {
6    stroke(0);
7    strokeWeight(12);
8
9    // Top-left
10   fill(255, 255, 255);
11   rect(0, 0, width, height);
12
13   // Top-middle
14   fill(0,0,255);
```



Narration

Storing and Retrieving Data

Narration

Exercise 1: Routine

- Going back to the first step of exercise 1:
  - Write down a daily routine, step by step
  - You go through these steps every day, in the same order
- How can we apply this to programming?

Narration

Imperative programming

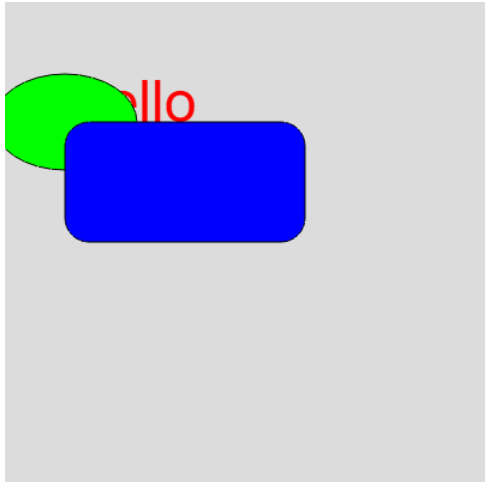
- Programming is like a routine
- You tell the computer what to do, step by step
- The computer evaluates the code line by line, top to bottom

Narration

Overlapping

- When drawing, new shapes go over old shapes
- The order of shapes matter

```
function setup() {
    createCanvas(400, 400);
    background(220); // grey background

    // Set text properties
    textSize(48); // big text
    fill(255, 0, 0); // red colour
    text("Hello", 50, 100);

    // Draw an ellipse
    fill(0, 255, 0); // green colour
    ellipse(50, 100, 120, 80);

    // Draw a rounded rectangle
    fill(0, 0, 255); // blue colour
    rect(50, 100, 200, 100, 20); // The 5th argument is *optional* and defines the corner
```

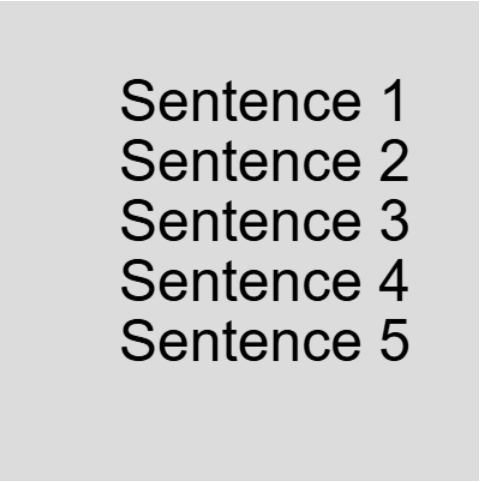


Narration

Drawing multiple lines of text

- Per line we can use a `text()` call
- We need to manually space these `50` pixels vertically apart

```
1  function setup() {
2      createCanvas(400, 400);
3      background(220); // grey background
4
5      // Set text properties
6      textSize(48); // big text
7
8      text("Sentence 1", 100, 100);
9      text("Sentence 2", 100, 150);
10     text("Sentence 3", 100, 200);
11     text("Sentence 4", 100, 250);
12     text("Sentence 5", 100, 300);
13 }
14
15 function draw() {
16     // No animation needed for this demonstration
```



Narration

Memorizing

- What if we want to draw more lines?
- We would need to write more `text()` calls and calculate the `y` position each time
- This is tedious and error-prone

Narration

Variables

- A **variable** is a named container for data

```
1  let interestingNumber = 42;
```

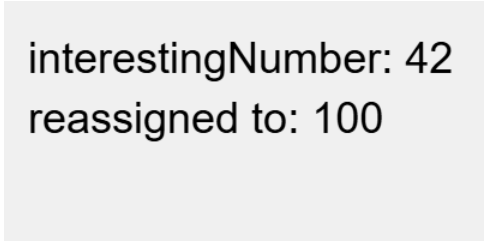
- Here we have used `let` to create a variable named `interestingNumber` that contains the number `42`
- We can now use `interestingNumber` in our code instead of having to write `42` every time
- You can store anything in a variable: numbers, text, etc.

Narration

Reassigning variables

- We can change the contents of a variable at any time
- This is called **reassigning** the variable
- We only use `let` once, when we first create the variable

```
1  function setup() {
2      createCanvas(400, 200);
3      background(240);
4
5      // Start with interestingNumber set to 42
6      let interestingNumber = 42;
7
8      // Draw the initial value as text
9      textSize(36);
10     text('interestingNumber: ' + interestingNumber, 20, 60);
11
12     // Reassign to a new value
13     interestingNumber = 100; // reassigned
```



Narration

Reassigning variables

- `interestingNumber = 100;` reassigns the variable to now contain `100`
- There are also shorthand operators to update number variables:
  - `interestingNumber++;` increases the variable by `1`
  - `interestingNumber--;` decreases the variable by `1`
  - `interestingNumber += 5;` increases the variable by `5`
  - `interestingNumber -= 2;` decreases the variable by `2`
  - `interestingNumber *= 3;` multiplies the variable by `3`
  - `interestingNumber /= 4;` divides the variable by `4`

Narration

Data Types: String

- We have seen the **string** data type before
- Strings are sequences of characters, enclosed in quotes `''` or `""`

```
1 let greeting = "Hello, world!";
```

Narration

Data Types: Integer

- An **integer** is a whole number (no decimal point)
- They can be positive, negative, or 0
- You can convert other types to integers using `int()`

```
let wholeNumber = 42;
let convertedNumber = int("123"); // converts string to integer
```

Narration

Data Types: Float

- A **float** (floating-point number) is a number with a decimal point
- They can also be positive, negative, or 0
- You can convert other types to floats using `float()`
- `round()`, `floor()`, and `ceil()` can round, round down, and round up floats to integers, respectively

```
let decimalNumber = 3.14;
let convertedFloat = float("3.14"); // converts string to float
let roundedNumber = round(3.7); // rounds to 4
```

Narration

Data Types: Boolean

- A **boolean** represents a logical value: `true` or `false`
- `true` and `false` are system keywords, similar to e.g. `mouseX` and `width`
- Booleans are often used in conditional statements to control the flow of a program

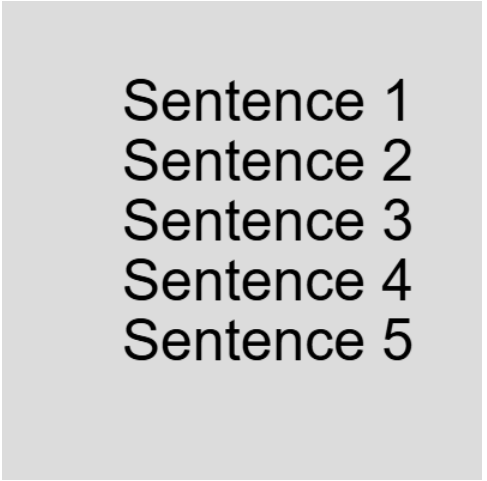
```
1 let isRaining = true;
2 let isSunny = false;
```

Narration

Using a variable for y position

- We set `verticalPosition` to `100` at the start
- Each time we draw a line of text, we increase `verticalPosition` by `50`
- If we want more lines we can copy-paste the two lines

```
function setup() {
    createCanvas(400, 400);
    background(220); // grey background

    // Set text properties
    textSize(48); // big text

    let verticalPosition = 100;
    text("Sentence 1", 100, verticalPosition);

    verticalPosition += 50;
    text("Sentence 2", 100, verticalPosition);

```



Narration

Building the example: variable ball

- Often we want to track the position of something
- Variables allow us to store and update this position over time

```
let x = 0;

function setup() {
    createCanvas(400, 400);
}

function draw() {
    background(220);

    // Draw a circle at position x
    fill(255, 0, 0);
    ellipse(x, 200, 50, 50);

    // Move the circle
    x += 2;

```


Narration

Logic

Narration

Conditional Statements

- Sometimes we want to only do something if a certain condition is met
  - E.g. reset the ball when it leaves the screen
- This is done with **conditional statements**

Narration

Conditions

- A **condition** is an expression that is `true` or `false` (boolean)
- We can use comparison operators to create conditions:
  - `==` (equal to)
  - `!=` (not equal to)
  - `<` (less than)
  - `>` (greater than)
  - `<=` (less than or equal to)
  - `>=` (greater than or equal to)

Narration

If statement

- An **if statement** checks a condition
- If the condition is true, the code inside the if statement is executed
- In the example below, we check whether `mouseX` is smaller than `200`.

```
1  function draw() {
2      background(220);
3      fill(0, 0, 255); // blue
4
5      // Change color based on mouse position
6      if (mouseX < 200) {
7          fill(255, 0, 0); // red
8      }
9
10     ellipse(mouseX, mouseY, 50, 50);
11 }
```

Narration

If statement

- The evaluated condition is between parentheses ( )
- The code that is executed on true is between brackets { }
- Indentation is not required, but makes code easier to read

```
1  function draw() {
2      background(220);
3      fill(0, 0, 255); // blue
4
5      // Change color based on mouse position
6      if (mouseX < 200) {
7          fill(255, 0, 0); // red
8      }
9
10     ellipse(mouseX, mouseY, 50, 50);
11 }
```

Narration

If-else statement

- What happens if the condition is false?
- We can use an **if-else statement** to provide alternative code to execute
- True -> do the first thing. False -> do the second thing.

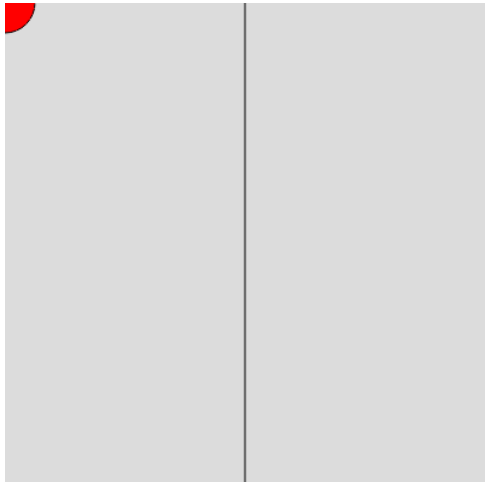
```
function draw() {
    background(220);

    // Change color based on mouse position
    if (mouseX < 200) {
        fill(255, 0, 0); // red
    } else {
        fill(0, 0, 255); // blue
    }

    ellipse(mouseX, mouseY, 50, 50);
}
```

Narration

Example: mouse circle

- Change the circle color based on mouse position
- If `mouseX < 200` -> red
- Else -> blue

```
1  function setup() {
2      createCanvas(400, 400);
3  }
4
5  function draw() {
6      background(220);
7
8      // Draw a line dividing the canvas
9      stroke(0);
10     line(200, 0, 200, 400);
11
12     // Change color based on mouse position
13     if (mouseX < 200) {
```

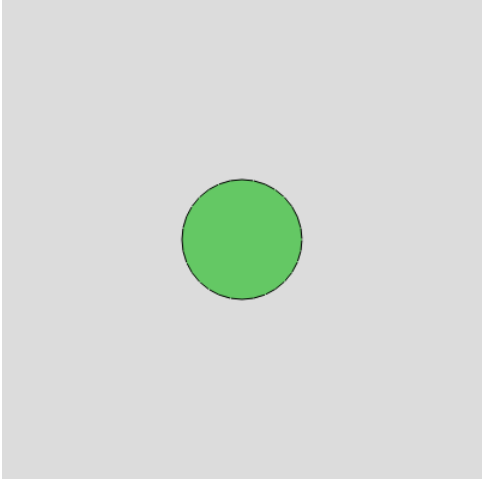


Narration

Example: two shapes

- Draw a shape based on vertical mouse position
- If `mouseY < 200` -> draw ellipse
- Else -> draw rectangle

```
1  function setup() {
2      createCanvas(400, 400);
3  }
4
5  function draw() {
6      background(220);
7
8      fill(100, 200, 100);
9
10     // Draw different shapes based on mouse y position
11     if (mouseY < 200) {
12         ellipse(200, 200, 100, 100);
13     } else {
```

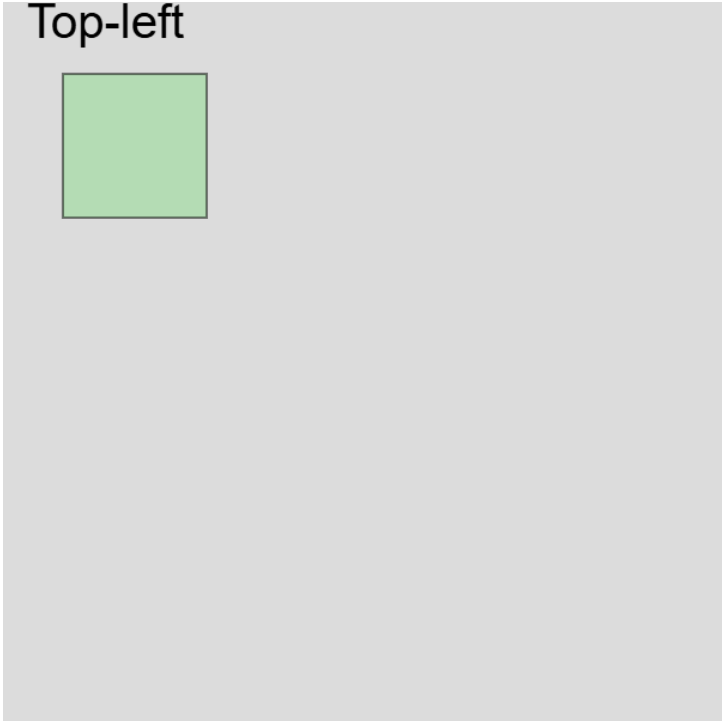


Narration

Nesting

- We can use if statements inside other if statements: **nesting**
- This allows us to check multiple conditions in a structured way

```
function setup() {
    createCanvas(600, 600);
}

function draw() {
    background(220);
    // Nested if: decide which quadrant the mouse is in
    textSize(40);
    fill(0);

    if (mouseX < width / 2) {
        // Left half
        if (mouseY < height / 2) {
            // Top-left
            text('Top-left', 20, 30);
            fill(180, 220, 180);
```



Narration

Conditional operators

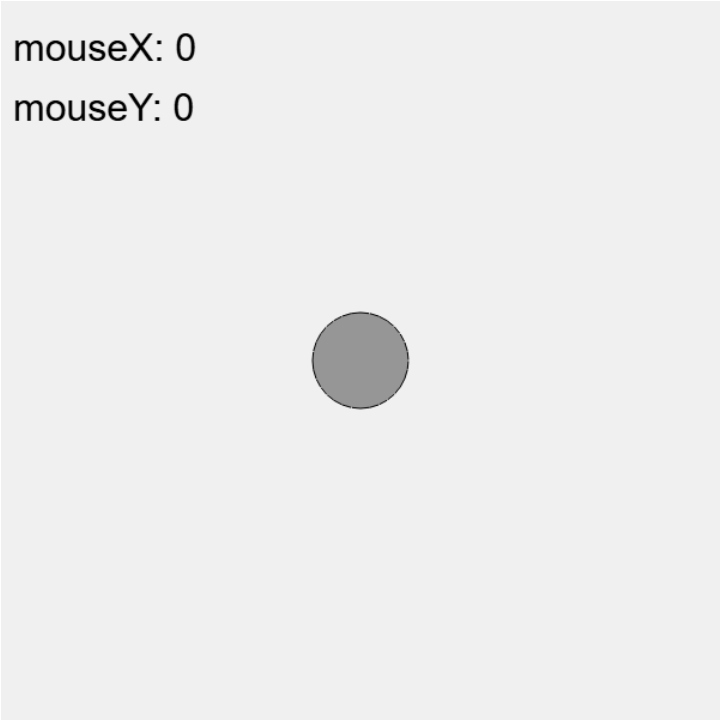
- We can combine conditions using **conditional operators**
- Common operators:
  - `&&` (and): both conditions must be true
  - `||` (or): at least one condition must be true

Narration

If else if

- We can chain multiple conditions using **if else if** statements
- This allows us to check several conditions in sequence

```
1  function setup() {
2      createCanvas(600, 600);
3  }
4
5  function draw() {
6      background(240);
7
8      // Show current mouse coordinates
9      fill(0);
10     textSize(32);
11     text('mouseX: ' + mouseX, 10, 50);
12     text('mouseY: ' + mouseY, 10, 100);
13
14     if ((mouseX > width / 2) && (mouseY > height / 2)) {
15         // If mouse bottom right
16         // AND true -> green circle
```

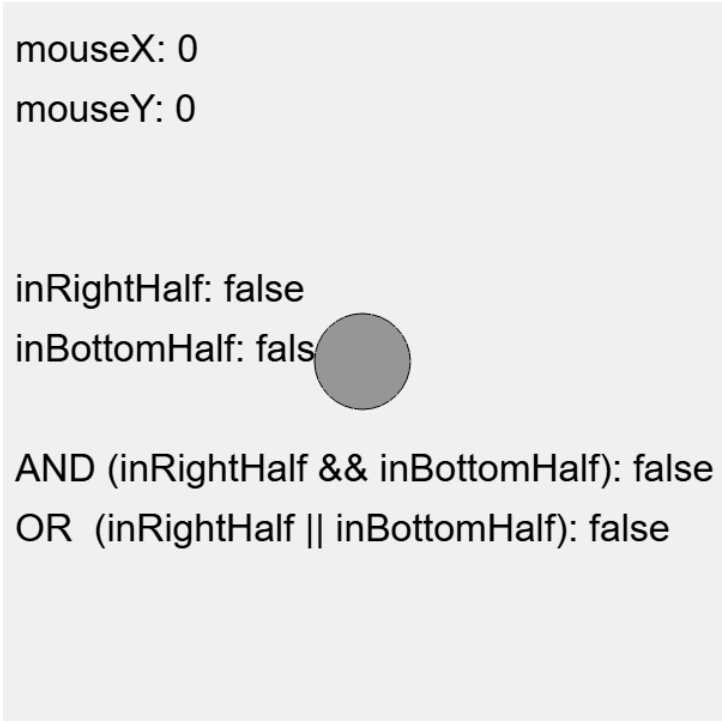


Narration

Storing conditions

- We can store the result of a condition in a variable
- This can make code easier to read and manage

```
1  function setup() {
2      createCanvas(600, 600);
3  }
4
5  function draw() {
6      background(240);
7
8      // Show current mouse coordinates
9      fill(0);
10     textSize(32);
11     text('mouseX: ' + mouseX, 10, 50);
12     text('mouseY: ' + mouseY, 10, 100);
13
14     // Store conditions in variables
15     let inRightHalf = mouseX > width / 2;
16     let inBottomHalf = mouseY > height / 2;
```

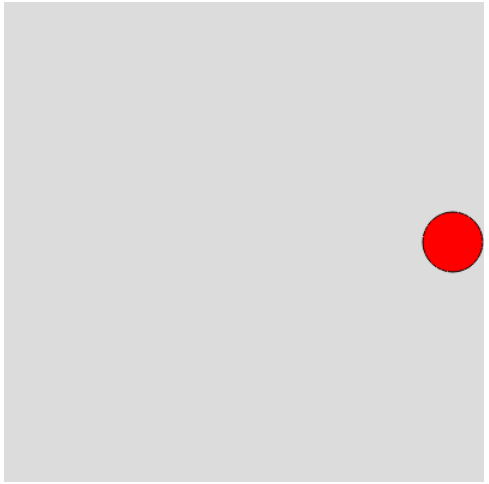


Narration

Building the example: resetting ball

- We can use conditions to check if the ball goes offscreen
- If it does, we reset its position

```
let x = 0;

function setup() {
    createCanvas(400, 400);
}

function draw() {
    background(220);

    // Draw a circle at position x
    fill(255, 0, 0);
    ellipse(x, 200, 50, 50);

    // Move the circle
    x += 2;

```

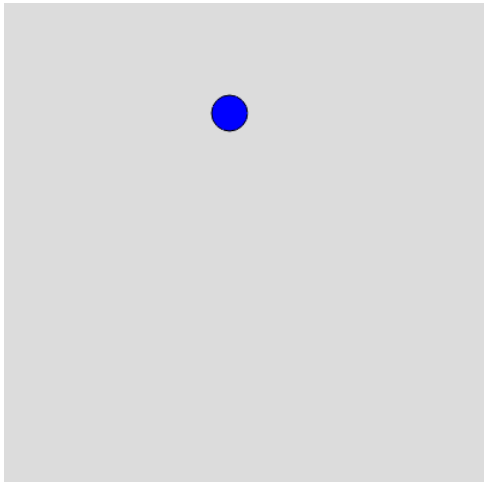


Narration

Building the example: bouncing ball

- With variables and conditions, we can make the ball bounce off the edges
- Instead of resetting the position, reverse the direction when hitting an edge

```
let x = 200;
let y = 200;
let speedX = 3;
let speedY = 2;
let diameter = 30;

function setup() {
    createCanvas(400, 400);
}

function draw() {
    background(220);

    // Draw the ball
```



Narration

Looping

Narration

Repeating lines of code

- Remember our text example; we want to draw many lines of text
- Copy-pasting code is tedious and error-prone
- We can use **loops** to repeat code multiple times

Narration

For loop

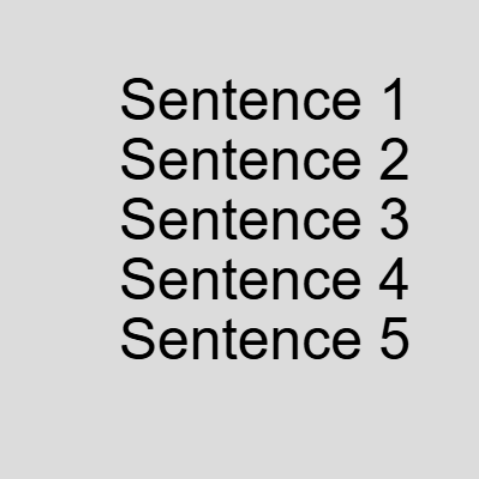
- A **for loop** repeats a block of code a specified number of times
- `for (initialization; condition; update) { // code to repeat }`
  - **initialization**: set up a variable to track iterations
  - **condition**: when to stop looping
  - **update**: change the variable each iteration

Narration

Lines example

- By putting our text drawing code inside a for loop, we can draw many lines easily
- In this example, we loop with variable `i` from `1` to `5`

```
function setup() {
    createCanvas(400, 400);
    background(220); // grey background

    // Set text properties
    textSize(48); // big text

    let verticalPosition = 100;

    for(let i = 1; i <= 5; i++) { // Loop from 1 to 5
        text("Sentence " + i, 100, verticalPosition);
        verticalPosition += 50;
    }
}
```

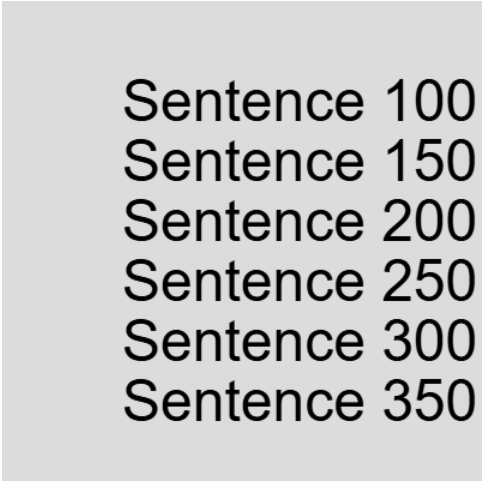


Narration

Looping with different update

- We can also use a different update step so that we do not need 2 variables

```
function setup() {
    createCanvas(400, 400);
    background(220); // grey background

    // Set text properties
    textSize(48); // big text

    for(let verticalPosition = 100; verticalPosition <= 350; verticalPosition += 50) { // Loop from 1 to 5
        text("Sentence " + verticalPosition, 100, verticalPosition);
    }
}

function draw() {
    // No animation needed for this demonstration
}
```



Narration

Scope

- **Scope**: Variables defined inside a loop are only accessible inside that loop
- Similarly, variables defined in `draw()` are not accessible in `setup()`
- You can define variables outside of functions to give them a broader scope

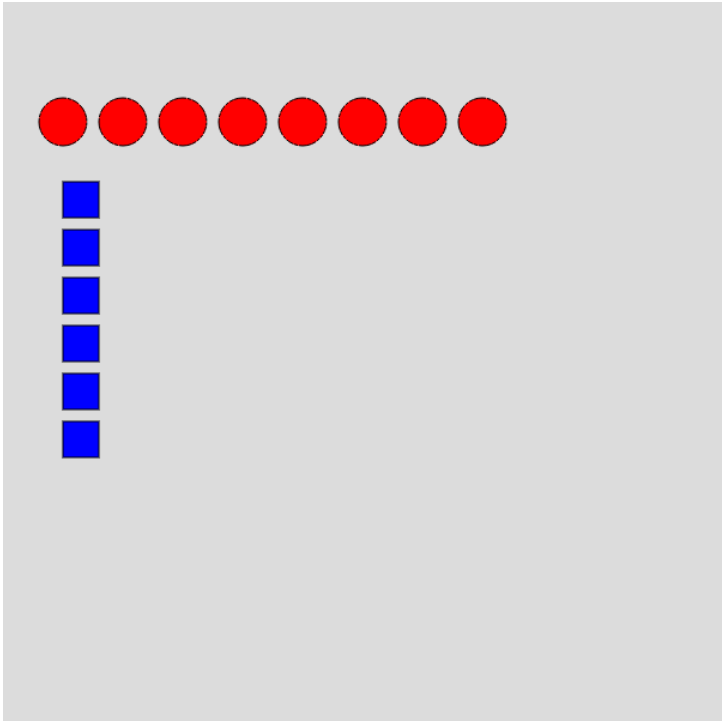
```
let firstVar = 5; // global (everywhere) scope

function setup() {
    let secondVar = 10; // local to setup
}

function draw() {
    let thirdVar = 20; // local to draw
}
```

Narration

Looping and scope

- In this example, the variable `i` is contained within the loop
- We can thus define a new variable `i` elsewhere without conflict

```
1  function setup() {
2      createCanvas(600, 600);
3      background(220);
4
5      // Draw a row of circles using a loop
6      for (let i = 0; i < 8; i++) {
7          fill(255, 0, 0);
8          ellipse(50 + i * 50, 100, 40, 40);
9      }
10
11     // Draw a column of squares using a loop
12     for (let i = 0; i < 6; i++) {
13         fill(0, 0, 255);
14         rect(50, 150 + i * 40, 30, 30);
15     }
16 }
```

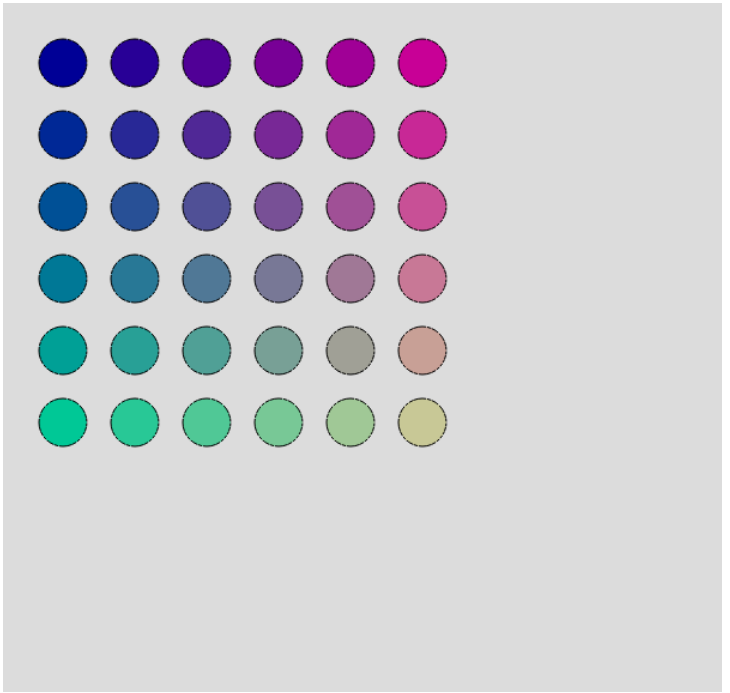


Narration

Nesting loops

- Similar to if statements, we can nest loops inside other loops
- This allows us to create more complex patterns and structures, usually 2D ones

```
1  function setup() {
2      createCanvas(600, 600);
3      background(220);
4
5      // Draw a grid of circles using nested loops
6      for (let i = 0; i < 6; i++) {
7          for (let j = 0; j < 6; j++) {
8              fill(i * 40, j * 40, 150);
9              ellipse(50 + i * 60, 50 + j * 60, 40, 40);
10         }
11     }
12 }
13
14 function draw() {
```



Narration

Arrays

Narration

Storing multiple values

- Sometimes we want to store multiple values
- We can use an **array** to store a list of values in a single variable
- An array is created using square brackets `[]`, with values separated by commas `,`

```
1  let colours = ['red', 'green', 'blue'];
```

- Here, `colours` is an array containing three strings: ‘red’, ‘green’, and ‘blue’

Narration

Accessing array elements

- We can access individual elements in an array using their index
- Array indices start at `0`
- To access the first element, we use `arrayName[0]`, the second element is `arrayName[1]`, and so on

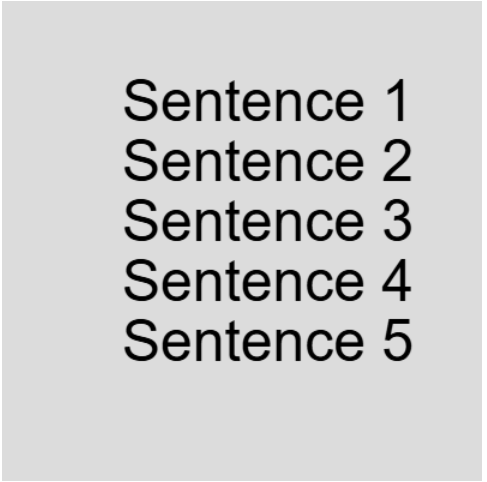
```
let colours = ['red', 'green', 'blue'];
let firstColour = colours[0]; // 'red'
let secondColour = colours[1]; // 'green'
```

Narration

Arrays of text

- Revisiting our text lines example, we can store the lines in an array
- This allows us to easily manage and access the content of the lines

```
function setup() {
    createCanvas(400, 400);
    background(220); // grey background

    // Set text properties
    textSize(48); // big text

    // Define an array of strings
    let sentences = ["Sentence 1", "Sentence 2", "Sentence 3", "Sentence 4", "Sentence 5"];

    // Access array elements by index
    text(sentences[0], 100, 100);
    text(sentences[1], 100, 150);
    text(sentences[2], 100, 200);
    text(sentences[3], 100, 250);
    text(sentences[4], 100, 300);
```



Narration

Using arrays

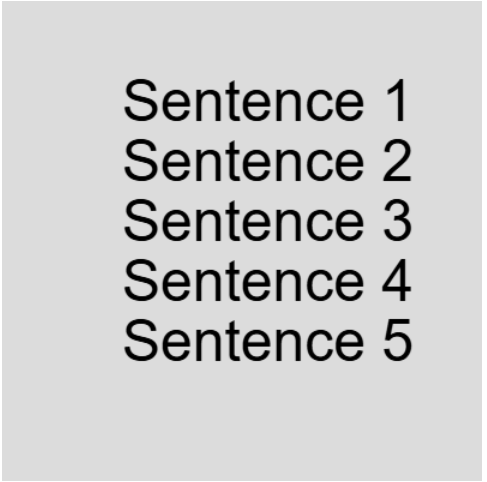
- Arrays have some useful properties and methods, such as
  - `myArray.length` gives the number of elements in the array
  - `myArray.push(value)` adds a new value to the end of the array
  - `myArray.pop()` removes the last value from the array
  - `myArray.sort()` sorts the array elements
  - `myArray.reverse()` reverses the order of the array elements
- Find more in the p5js reference

Narration

For loop + array

- Arrays and loops work well together
- We can use a for loop to iterate over the elements of an array

```
1  function setup() {
2      createCanvas(400, 400);
3      background(220); // grey background
4
5      // Set text properties
6      textSize(48); // big text
7
8      // Define an array of strings
9      let sentences = ["Sentence 1", "Sentence 2", "Sentence 3", "Sentence 4", "Sentence 5"];
10
11     // Use a for loop to iterate through the array
12     for (let i = 0; i < sentences.length; i++) {
13         text(sentences[i], 100, 100 + i * 50);
14     }
15 }
16
```



Narration

Nesting arrays

- Arrays can also contain other arrays, creating multi-dimensional arrays
- This is useful for storing grid-like data structures

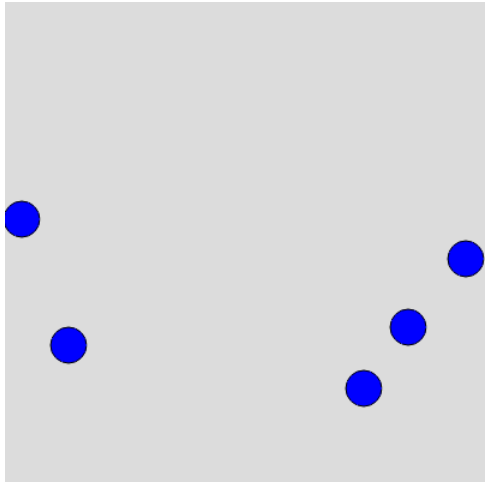
```
1   // Define a 2D array (grid)
2   let grid = [
3       [1, 2, 3],
4       [4, 5, 6],
5       [7, 8, 9]
6   ];
7
8   function setup() {
9       createCanvas(300, 300);
10  }
11
12  function draw() {
13      background(220);
14      fill(0);
15      textSize(32);
16
```

Narration

Building the example: bouncing ball

- Using arrays we can define multiple balls
- We can use a for loop to update and draw all balls

```
let x = [200, 100, 300, 150, 250];
let y = [200, 100, 300, 150, 250];
let speedX = [3, -2, 4, -1, 2];
let speedY = [2, 3, -2, 4, -3];
let diameter = 30;

function setup() {
    createCanvas(400, 400);
}

function draw() {
    background(220);

    // Go over every ball
    for (let i = 0; i < 5; i++) {
        // Draw the ball
```

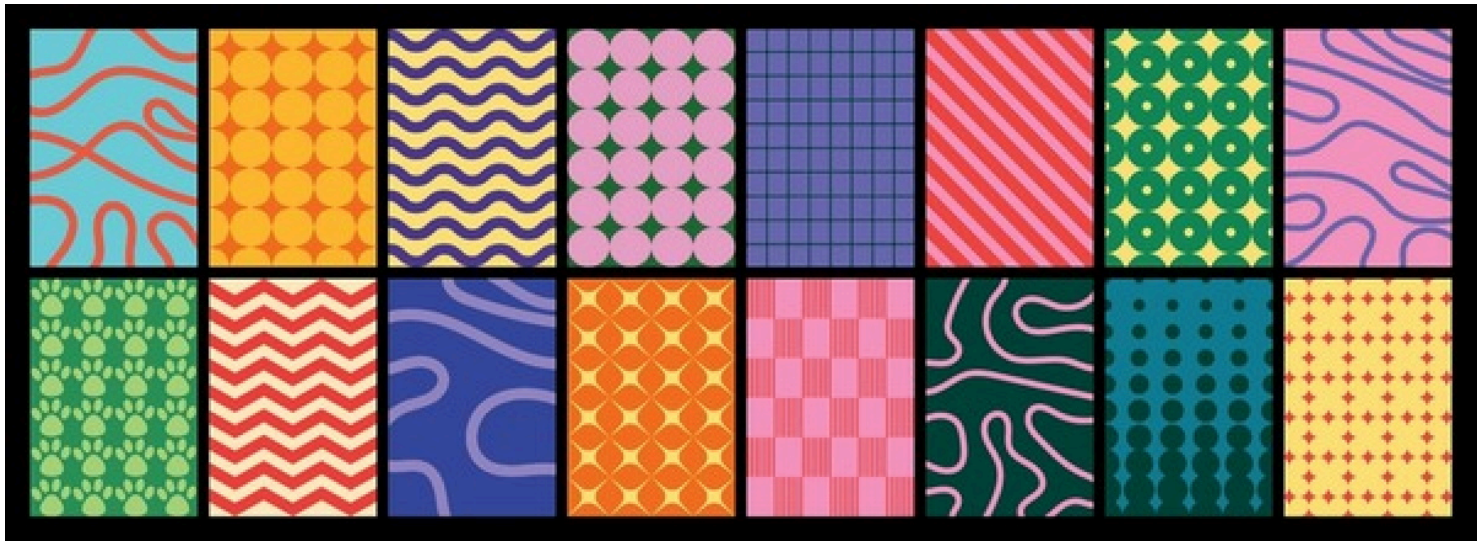


Narration

Put arrays and loops to the test!

Class Assignment: Patterns!

1. In a group of 1-3, start by drawing some patterns on paper or a whiteboard (e.g. https://www.tldraw.com)
2. Then, implement these patterns in p5js using loops and arrays where appropriate.
3. Can you make them interactive using mouseX and mouseY?



Narration

Summary

- We covered variables, data types, conditional statements, loops, and arrays
- These are fundamental programming concepts that will help you create more complex and interactive sketches
- Practice these concepts; it is a lot for one week, do not be afraid to make mistakes.
- Project groups will be published today.

Narration